

The absint package

Florian Sihler

Version 2.0

Notation for abstract interpretation: lattices, domains, Galois connections, widening and narrowing, denotational semantics.

1 Usage

`\absexpr` switches to absint notation and works in text and in math mode:

```
\absexpr{(X, \partof, \lub, \glb, \bot, \top)}
(X, \sqsubseteq, \sqcup, \sqcap, \perp, \top)
```

For longer parts and for display math use the absint environment:

```
1 \begin{absint}
2   \[ \Lub_{i \in \mathbb{N}} f^i(\bot) \partof \lfp_{\bot} f \]
3 \end{absint}
```

$$\bigsqcup_{i \in \mathbb{N}} f^i(\perp) \sqsubseteq \text{lfp}_{\perp} f$$

Outside of the two, `\lor` and `\sqcup` keep their usual meaning. Names that are still free (`\lub`, `\Environments`, `\Interval`, ...) are defined for plain math mode too, see stubs. `\absintuse{lub}` typesets a single symbol by name: $\sqcup$.

2 Concrete and abstract

`\abstract` and `\concrete` mark one object: $X^\# \sqsubseteq Y$. A whole expression switches with `\abstractly` and `\concretely`:

```
\absexpr{\abstractly{(X, \partof, \lub, \glb)}}
(X, \sqsubseteq^\#, \sqcup^\#, \sqcap^\#)
```

Only symbols that have an abstract form change; Table 1 shows both forms of those that do. `\isAbstract` appends the mark to whatever precedes it, `\sharp` chooses it.

3 Options

Options go to `\usepackage`, or later to `\absintsetup`:

```
1 \absintsetup{color=teal, sharp=\sharp}
```

color	colour of the notation. purple.
colored	colour at all. true.
mathcolor	colour with <code>\mathcolor</code> instead of a math group, which also catches the scripts that follow a symbol but is markedly slower. false.

style	one argument macro applied to every symbol, replacing the colouring. For more, redefine <code>\absintstyle</code> .
links	link a symbol to its legend entry. true, off if hyperref is missing. A symbol without an entry stays plain.
labelprefix	prefix of the placed labels, absint: by default.
track	record the used symbols in the .aux file, needed by <code>\absintlegend</code> . true.
sharp	mark of the abstract world. \#, often <code>\sharp</code> .
flat	mark of the concrete world, appended by <code>\concrete</code> . Empty.
brackets	semantic brackets: stmaryrd loads that package, fallback builds them from two brackets, auto takes <code>\llbracket</code> if something else provides it. auto.
opfont	font of the named operators. <code>\mathsf</code> .
domainfont	font of the domains. <code>\mathbb</code> .
calfont	font of the calligraphic domains. <code>\mathcal</code> .
substitution	how <code>\subst</code> is set: assign gives $M[x := N]$, slash gives $M[N/x]$. assign.
stubs	define the free names for plain math mode as well. Also what lets <code>\absexpr</code> survive a <code>\section</code> title. true.
shortcut	another name for <code>\absexpr</code> , without its backslash: shortcut=ai defines <code>\ai</code> .
plain	short for colored=false, links=false.

4 Legend and references

`\absintlegend` lists the notation used in the document and places the label of each entry, which is what the symbols link to. It settles after two to three runs and says when another one is due. `\absintlegend*` lists everything and `\absintcommands` the commands.

`\absintmark{lub,glb}` places the labels by hand instead; those win over the legend. The layout comes from these hooks:

absintlegendstart	before the list.
absintlegendstop	after the list.
absintlegendentry	symbol, name, description, aliases.
absintcommandentry	example, name, description, source.
absintlegendempty	instead of the list when there is nothing to list.
absintname	a name.
absintdemo	the source of an example.

`\absintforallsymbols` and `\absintforallcommands` apply a one argument macro to every name. This manual uses them to highlight the notation and to lay out Tables 1 and 2.

5 Lambda calculus

Abstraction is `\lam`, application is `\app` and everything else reads the way it is written:

```

1 \absexpr{(\lam{x}{M}) \app N \betared \subst{M}{x}{N}}
2 \absexpr{\lam{x}{M \app x} \etared M \text{ if } x \notin \text{FV}\{M\}}
3 \absexpr{\comby \app f \betareds f \app (\comby \app f)}

```

$(\lambda x. M) N \longrightarrow_{\beta} M[x := N]$, $\lambda x. M x \longrightarrow_{\eta} M$ if $x \notin \text{FV}(M)$, $\forall f \rightarrow_{\beta} f (\forall f)$

Typing has `\typing`, `\ctxadd` and the usual arrows, so a rule premise is one line:

```

1 \absexpr{\typing{\ctxadd{\Context}{x}{\sigma}}{M}{\tau}}
2 \implies \typing{\Context}{\lam{x}{M}}{\sigma \to \tau}

```

$\Gamma, x : \sigma \vdash M : \tau \implies \Gamma \vdash \lambda x. M : \sigma \rightarrow \tau$

and the typed and polymorphic abstractions are `\lamty`, `\tyabs` and `\forallallty`: $\Lambda X. \lambda x : X. x : \forall X. X \rightarrow X$.

6 Own notation

```
1 \DeclareAbsintSymbol{signs}{\mathcal{S}}
2 [sharp, aliases={Sign}, desc={the sign domain}]
3 \DeclareAbsintCommand[desc={a widening with a threshold}, demo={\widenat{k}}]
4 {\widenat}[1]{\widen_{#1}}
```

gives $\mathcal{S}$, $\mathcal{S}^\#$ and $a \nabla_k b$. An empty meaning lifts the command of that name, which is how the standard symbols are promoted: `\DeclareAbsintSymbol{lor}{\lor}[bin]`.

class the math class: ord, bin, rel, op, open, close, punct or inner, written as `class=bin` or just `bin`. Keeps the spacing right, so set it. `ord`.

abstract rendering in the abstract world.

sharp short for an abstract with the mark appended.

aliases further names, sharing rendering and legend entry.

desc description for the legend.

`\DeclareAbsintAlias` adds a single alias later on. `\DeclareAbsintCommand` is `\newcommand`, optional arguments included, for a command that only exists in absint notation; its `desc` and `demo` feed `\absintcommands`.

7 The notation

Symbols with two forms are shown concrete first, abstract second.

Table 1: The symbols

Symbol	Name	Description	Aliases
$\neg$	<code>\lnot</code>	negation	<code>\neg</code>
$\wedge$	<code>\land</code>	conjunction	<code>\wedge</code>
$\vee$	<code>\lor</code>	disjunction	<code>\vee</code>
$\Rightarrow$	<code>\implies</code>	implication	
$\Leftrightarrow$	<code>\iff</code>	bi-implication	
$\forall$	<code>\forallall</code>	universal quantifier	
$\exists$	<code>\existsists</code>	existential quantifier	
$\nexists$	<code>\nexists</code>	there is no	
$=$	<code>\eq</code>	equality	
$\neq$	<code>\neq</code>	inequality	
$\stackrel{\text{def}}{=}$	<code>\defeq</code>	equality by definition	
$\emptyset$	<code>\emptyset</code>	the empty set	
$\in$	<code>\in</code>	membership	
$\notin$	<code>\notin</code>	non-membership	
$\ni$	<code>\ni</code>	contains	
$\subset$	<code>\subset</code>	strict subset	
$\subseteq$	<code>\subseteq</code>	subset	
$\supset$	<code>\supset</code>	strict superset	
$\supseteq$	<code>\supseteq</code>	superset	
$\cup$, $\cup^\#$	<code>\cup</code>	union	<code>\union</code>
$\cap$, $\cap^\#$	<code>\cap</code>	intersection	<code>\intersect</code>
$\setminus$	<code>\setminus</code>	set difference	
$\times$	<code>\times</code>	cartesian product	

Symbol	Name	Description	Aliases
$\bigcup, \bigcup^\#$	<code>\bigcup</code>	union of a family	<code>\bigunion</code>
$\bigcap, \bigcap^\#$	<code>\bigcap</code>	intersection of a family	<code>\bigintersect</code>
$\mathcal{P}, \mathcal{P}^\#$	<code>\powerset</code>	powerset	<code>\P</code>
$\mathcal{P}_{\text{fin}}$	<code>\finpowerset</code>	powerset restricted to the finite subsets	
$\sqsubseteq, \sqsubseteq^\#$	<code>\sqsubseteq</code>	partial order	<code>\partof</code>
$\sqsubset, \sqsubset^\#$	<code>\sqsubset</code>	strict partial order	<code>\strictpartof</code>
$\sqsupseteq, \sqsupseteq^\#$	<code>\sqsupseteq</code>	inverse partial order	
$\sqsupset, \sqsupset^\#$	<code>\sqsupset</code>	strict inverse partial order	
$\leq$	<code>\leq</code>	less than or equal	<code>\le</code>
$\geq$	<code>\geq</code>	greater than or equal	<code>\ge</code>
$\sqcup, \sqcup^\#$	<code>\sqcup</code>	least upper bound (join)	<code>\lub, \join</code>
$\sqcap, \sqcap^\#$	<code>\sqcap</code>	greatest lower bound (meet)	<code>\glb, \meet</code>
$\bigsqcup, \bigsqcup^\#$	<code>\bigsqcup</code>	least upper bound of a family	<code>\Lub, \Join</code>
$\bigsqcap, \bigsqcap^\#$	<code>\bigsqcap</code>	greatest lower bound of a family	<code>\Glb, \Meet</code>
$\top, \top^\#$	<code>\top</code>	greatest element	
$\bot, \bot^\#$	<code>\bot</code>	least element	
$\nabla, \nabla^\#$	<code>\widen</code>	widening	
$\triangle, \triangle^\#$	<code>\narrow</code>	narrowing	
$\rightarrow$	<code>\to</code>	function space	
$\mapsto$	<code>\mapsto</code>	maps to	
$\xrightarrow{\text{m}}$	<code>\monoto</code>	monotone function space	
$\circ$	<code>\compose</code>	function composition	
<code>id</code>	<code>\id</code>	the identity function	
<code>dom</code>	<code>\dom</code>	domain of a function	
<code>codom</code>	<code>\codom</code>	codomain of a function	
<code>lfp</code>	<code>\lfp</code>	least fixpoint	
<code>gfp</code>	<code>\gfp</code>	greatest fixpoint	
<code>Fix</code>	<code>\Fix</code>	the fixpoints of a function	
α	<code>\abstraction</code>	abstraction function	<code>\abstr</code>
γ	<code>\concretization</code>	concretization function	<code>\concr</code>
$\xleftrightarrow[\gamma]{\alpha}$	<code>\galois</code>	Galois connection	
$\xrightarrow[\gamma]{\alpha}$	<code>\galoisinsertion</code>	Galois insertion	
$\#$	<code>\abstractmark</code>	the abstraction mark	
$\mathbb{N}$	<code>\N</code>	the natural numbers	
$\mathbb{Z}$	<code>\Z</code>	the integers	
$\mathbb{Q}$	<code>\Q</code>	the rationals	
$\mathbb{R}$	<code>\R</code>	the reals	
$\mathbb{C}$	<code>\C</code>	the complex numbers	
$\mathbb{B}$	<code>\Bool</code>	the booleans	
$\mathbb{U}, \mathbb{U}^\#$	<code>\Universe</code>	the universe	
$\mathbb{V}, \mathbb{V}^\#$	<code>\Variables</code>	the variables	
$\mathbb{I}, \mathbb{I}^\#$	<code>\NumericDomain</code>	a numeric domain	
$\mathbb{B}, \mathbb{B}^\#$	<code>\BooleanDomain</code>	a boolean domain	
$\mathcal{I}, \mathcal{I}^\#$	<code>\IntervalDomain</code>	the interval domain	
$\mathcal{D}, \mathcal{D}^\#$	<code>\SetOfStates</code>	the states	
$\mathcal{E}, \mathcal{E}^\#$	<code>\Environments</code>	the environments	
$\rho, \rho^\#$	<code>\Environment</code>	an environment	
$\diamond, \diamond^\#$	<code>\AnyOp</code>	an arbitrary operator	

Symbol	Name	Description	Aliases
$\boxtimes, \boxtimes^\#$	<code>\AnyCompare</code>	an arbitrary comparison	
$\oplus, \oplus^\#$	<code>\intadd</code>	addition in a domain	
$\ominus, \ominus^\#$	<code>\intsub</code>	subtraction in a domain	<code>\intneg</code>
$\otimes, \otimes^\#$	<code>\intmul</code>	multiplication in a domain	
$\oslash, \oslash^\#$	<code>\intdiv</code>	division in a domain	
$\odot, \odot^\#$	<code>\intmod</code>	modulo in a domain	
$\mathbb{E}, \mathbb{E}^\#$	<code>\RawEvaluateExpr</code>	expression semantics	
$\mathbb{C}, \mathbb{C}^\#$	<code>\RawEvaluateCond</code>	condition semantics	
$\mathbb{S}, \mathbb{S}^\#$	<code>\RawEvaluateStatement</code>	statement semantics	
λ	<code>\lambda</code>	abstraction	
Λ	<code>\tylam</code>	type abstraction	
$\longrightarrow$	<code>\app</code>	application	
$\longrightarrow$	<code>\redto</code>	reduces to	
$\twoheadrightarrow$	<code>\redtos</code>	reduces to in many steps	
$\longrightarrow_\beta$	<code>\betared</code>	beta reduction	
$\twoheadrightarrow_\beta$	<code>\betareds</code>	beta reduction in many steps	
$=_\beta$	<code>\betaaeq</code>	beta equality	
$\longrightarrow_\eta$	<code>\etared</code>	eta reduction	
$=_\eta$	<code>\etaaeq</code>	eta equality	
$=_\alpha$	<code>\alphaaeq</code>	alpha equality	
$\longrightarrow_\delta$	<code>\deltared</code>	delta reduction	
$\Downarrow$	<code>\evalto</code>	evaluates to	
$\vdash$	<code>\types</code>	entails	<code>\entails</code>
$:$	<code>\hastype</code>	has type	
$<:, <:^\#$	<code>\subtype</code>	subtype of	
Λ	<code>\LambdaTerms</code>	the lambda terms	
$\mathcal{T}, \mathcal{T}^\#$	<code>\TypeSet</code>	the types	
$\Gamma, \Gamma^\#$	<code>\Context</code>	a typing context	
$\cdot$	<code>\emptyctx</code>	the empty context	
$[]$	<code>\hole</code>	the hole of a context	
I	<code>\combI</code>	the I combinator	
K	<code>\combK</code>	the K combinator	
S	<code>\combS</code>	the S combinator	
Y	<code>\combY</code>	the Y combinator	
$\mathcal{S}, \mathcal{S}^\#$	<code>\signs</code>	the sign domain	<code>\Sign</code>

Table 2: The commands

Example	Name	Description	Source
$a \sqcup^\# b$	<code>\abstractly</code>	switch to the abstract world	<code>\abstractly {a \lub b}</code>
$a \sqcup b$	<code>\concretely</code>	switch to the concrete world	<code>\concretely {a \lub b}</code>
$X^\#$	<code>\isAbstract</code>	the abstraction mark	<code>X\isAbstract</code>
X	<code>\isConcrete</code>	the concretion mark	<code>X\isConcrete</code>
$X^\#$	<code>\abstract</code>	mark one object as abstract	<code>\abstract {X}</code>
X	<code>\concrete</code>	mark one object as concrete	<code>\concrete {X}</code>
$a \stackrel{\text{def}}{=} b$	<code>\asdef</code>	by definition	<code>a \asdef \eq b</code>
$[e]$	<code>\semantics</code>	semantic brackets	<code>\semantics {e}</code>

Example	Name	Description	Source
$\mathbb{E}[e]\rho$	<code>\EvaluateExpr</code>	expression semantics	<code>\EvaluateExpr {e}\Environment</code>
$\mathbb{C}[b]\rho$	<code>\EvaluateCond</code>	condition semantics	<code>\EvaluateCond {b}\Environment</code>
$\mathbb{S}[s]\rho$	<code>\EvaluateStatement</code>	statement semantics	<code>\EvaluateStatement {s}\Environment</code>
$\{x \in \mathbb{N} \mid x \leq 3\}$	<code>\Set</code>	a set, <code>\Given</code> or <code>\For</code> as the separator	<code>\Set {x \in \mathbb{N} \Given x \leq 3}</code>
$\langle a, b \rangle$	<code>\Tuple</code>	a tuple	<code>\Tuple {a, b}</code>
$ X $	<code>\card</code>	cardinality	<code>\card {X}</code>
$[a, b]$	<code>\Interval</code>	an interval	<code>\Interval {a}{b}</code>
$f _S$	<code>\restrict</code>	restriction of a function	<code>\restrict {f}{S}</code>
$\rho[x \mapsto v]$	<code>\envupdate</code>	update of an environment	<code>\envupdate {\Environment }{x}{v}</code>
$\rho(x)$	<code>\envlookup</code>	lookup in an environment	<code>\envlookup {\Environment }{x}</code>
$\mathbb{Z} \overset{\alpha}{\underset{\gamma}{\rightleftarrows}} \mathbb{I}$	<code>\galoisbetween</code>	a Galois connection	<code>\galoisbetween {\mathbb{Z} }{\mathbb{I} \NumericDomain }</code>
$\lambda x. x x$	<code>\lam</code>	an abstraction	<code>\lam {x}{x \app x}</code>
$\lambda x : \mathcal{T}. x$	<code>\lamty</code>	a typed abstraction	<code>\lamty {x}{\TypeSet }{x}</code>
$\Lambda X. M$	<code>\tyabs</code>	a type abstraction	<code>\tyabs {X}{M}</code>
$\forall X. X \rightarrow X$	<code>\forallallty</code>	a universal type	<code>\forallallty {X}{X \to X}</code>
$M[x := N]$	<code>\subst</code>	a substitution, see the ‘substitution’ option	<code>\subst {M}{x}{N}</code>
$\text{FV}(M)$	<code>\FV</code>	the free variables	<code>\FV {M}</code>
$\text{BV}(M)$	<code>\BV</code>	the bound variables	<code>\BV {M}</code>
$\bar{3}$	<code>\church</code>	a Church numeral	<code>\church {3}</code>
$\Gamma \vdash M : \mathcal{T}$	<code>\typing</code>	a typing judgement	<code>\typing {\Context }{M}{\TypeSet }</code>
$\Gamma, x : \mathcal{T}$	<code>\ctxadd</code>	a context extended by a binding	<code>\ctxadd {\Context }{x}{\TypeSet }</code>
$C[M]$	<code>\ctxfill</code>	a context with its hole filled	<code>\ctxfill {C}{M}</code>
∇_k	<code>\widenat</code>	a widening with a threshold	<code>\widenat {k}</code>